

Software architecture

Software architecture

Rodrigo Martins Pagliares

Software development as model transformations

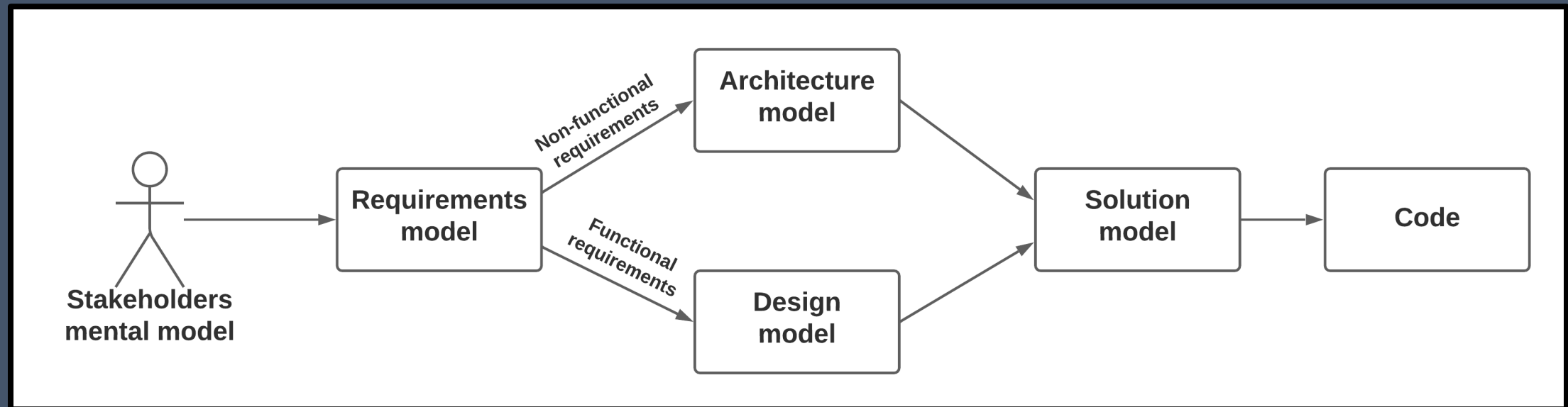
From stakeholder's mental model to code

Lecture: From stakeholder's mental model to code

Module: Software development as model transformations

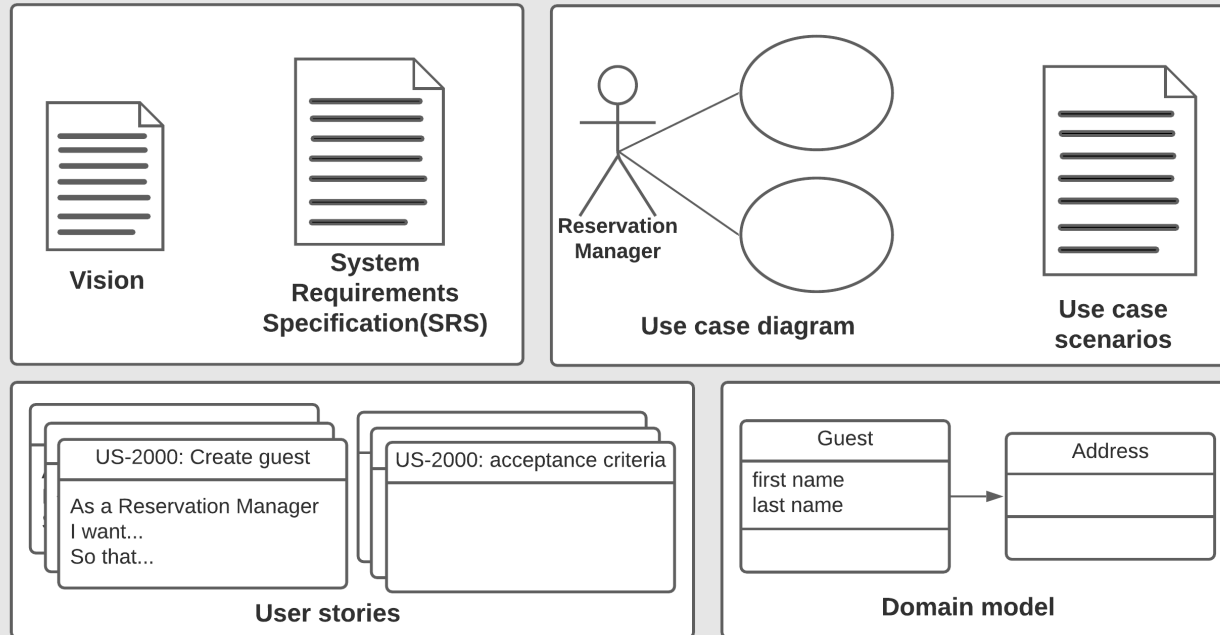
Software development models

- **Software development** can be viewed as a series of transformations from the **Stakeholder's mental model** to the **actual code**.



Requirements model

Requirements model



Use case scenario: Create a Guest

Pre-condition(s):

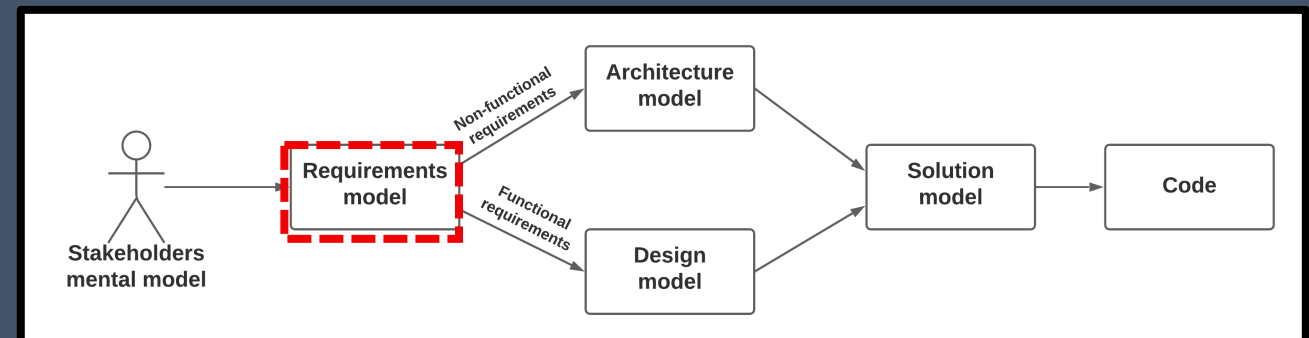
- A screen with an option "*Create new Guest*" is visible.
- The Reservation Manager is authenticated.

Main flow:

- 1.The Reservation Manager selects the option "*Create new Guest*".
- 2.The system show a form requesting Guest data.
- 3.The Reservation Manager enters the form data.
- 4.The systems shows a message "*Guest created sucessfully*".

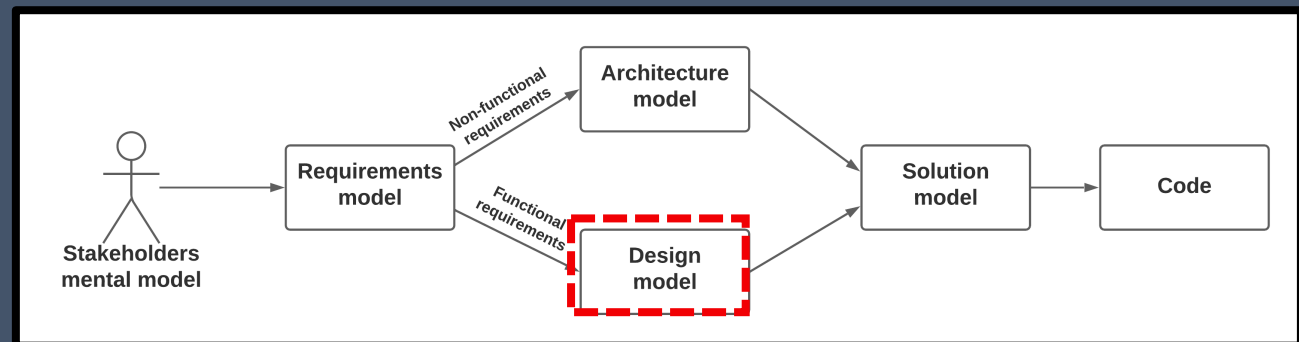
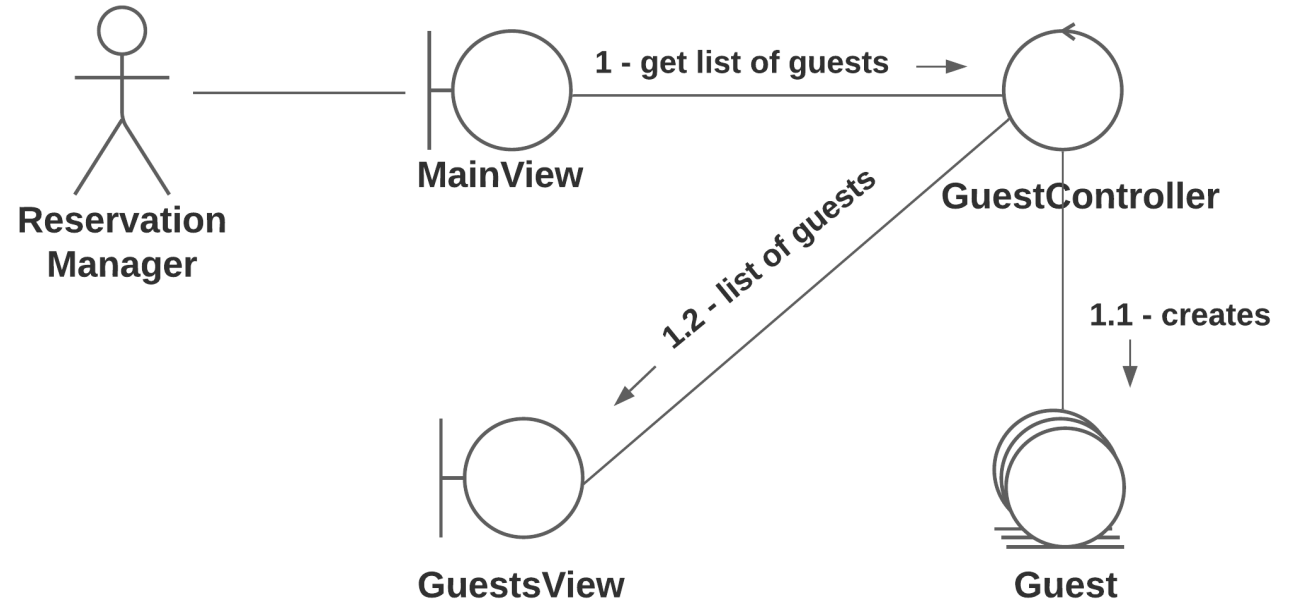
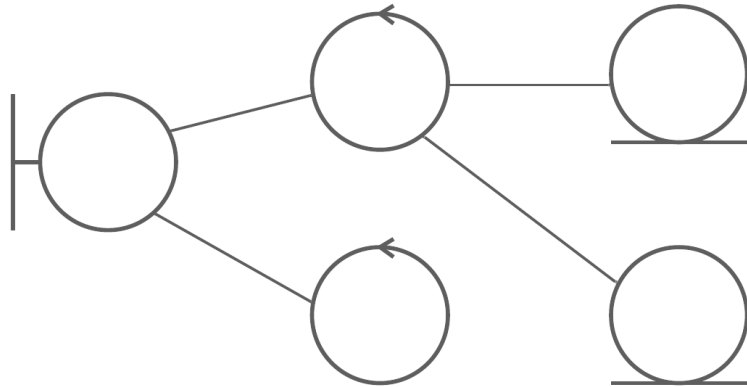
Alternate flow(s):

....
....

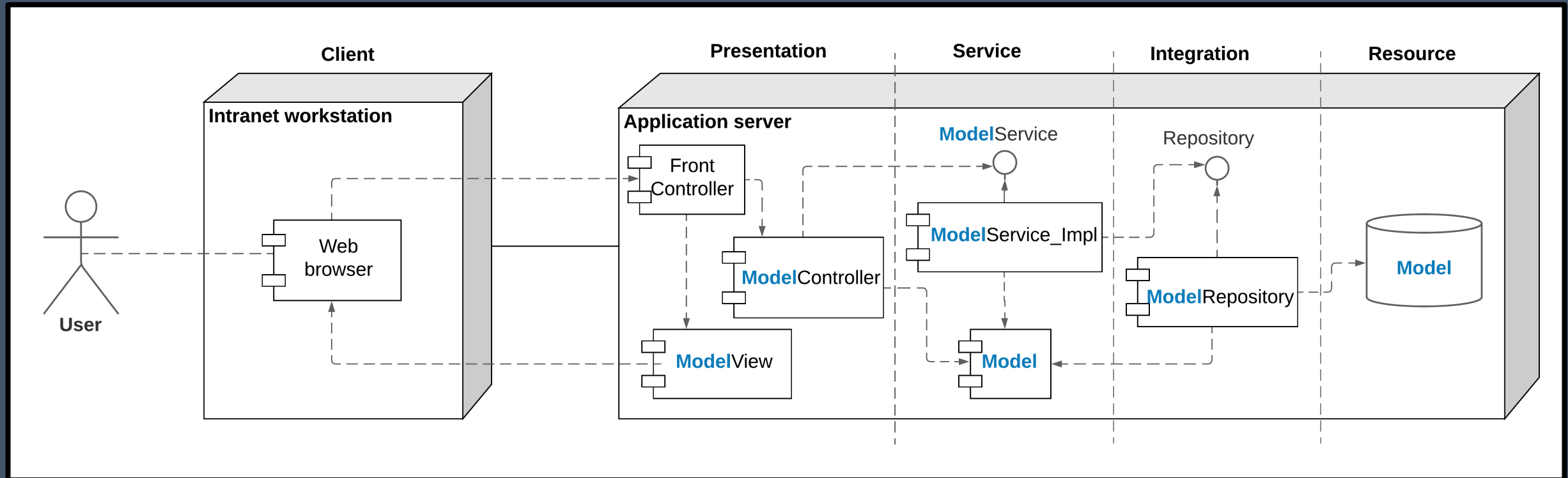
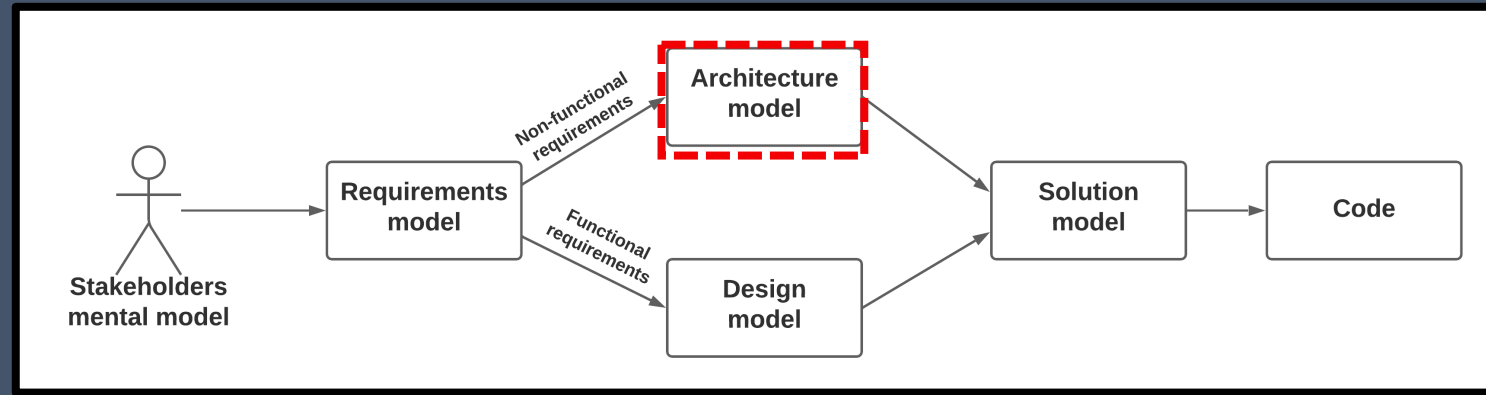


Design model

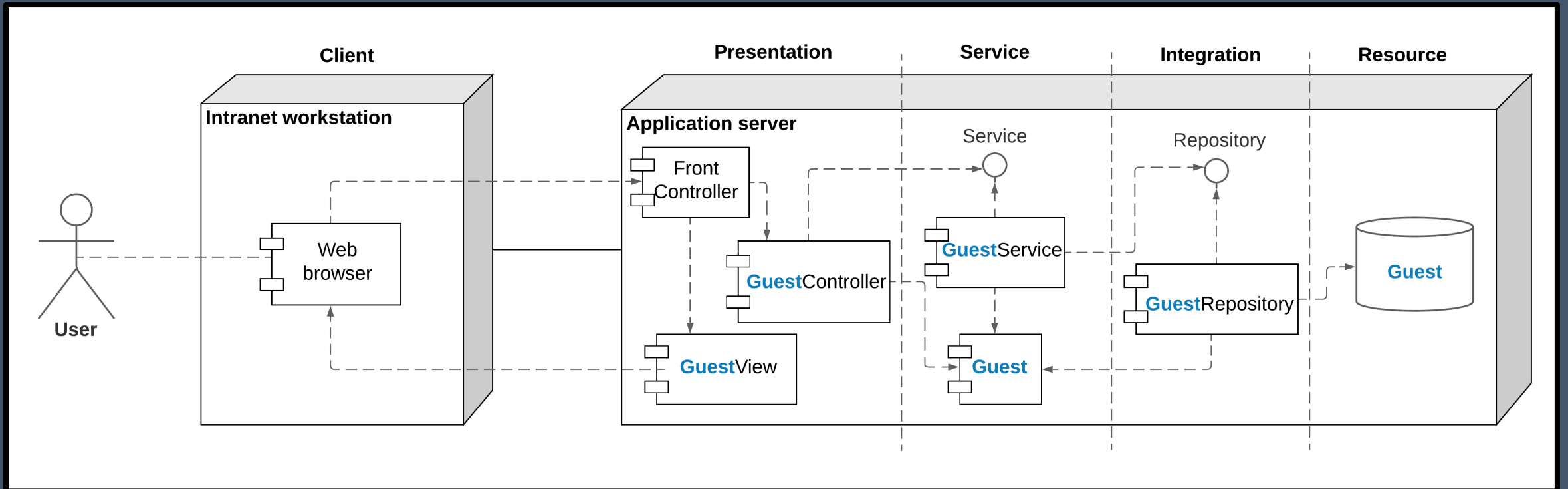
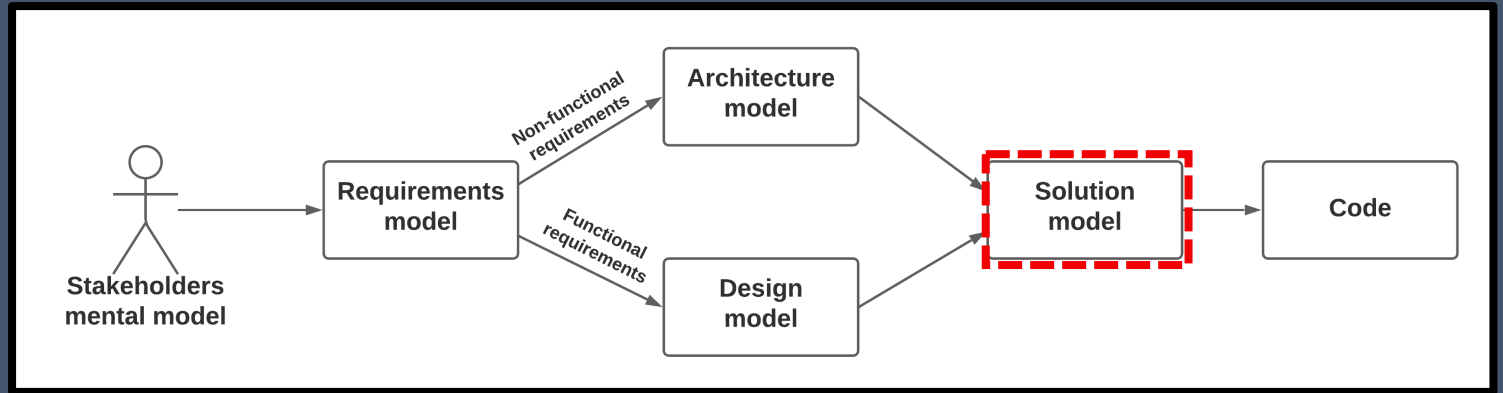
Design model



Architecture model



Solution model



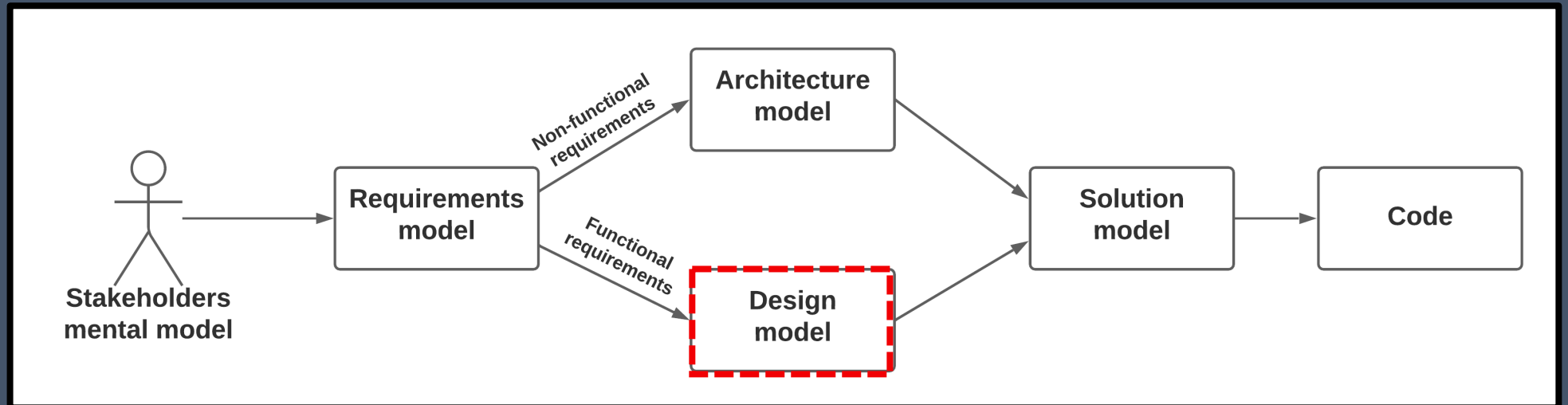
From a Requirements model to a Design model

Lecture: From a Requirements model to a Design model

Module: Software development as model transformations

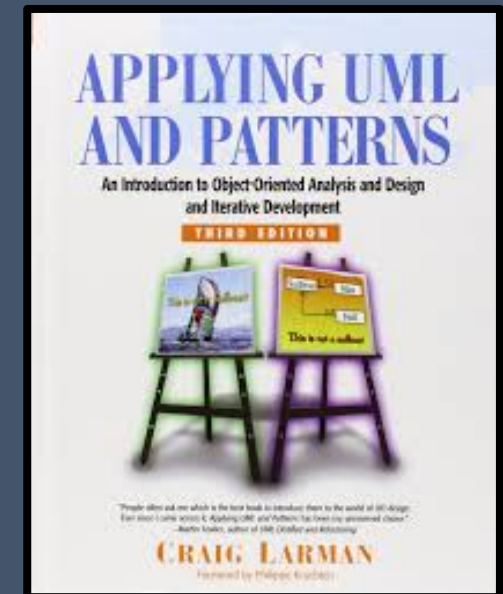
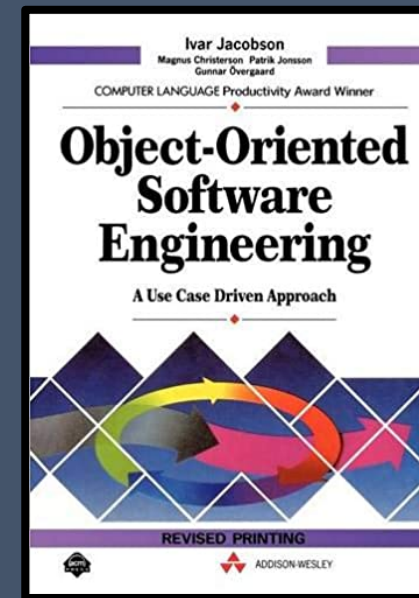
Design model

- Created from the **Requirements model**:
 - ✓ Use cases, user stories, Domain model, operation contracts, etc.
- Merged with the **Architecture model** to produce the **Solution model**.



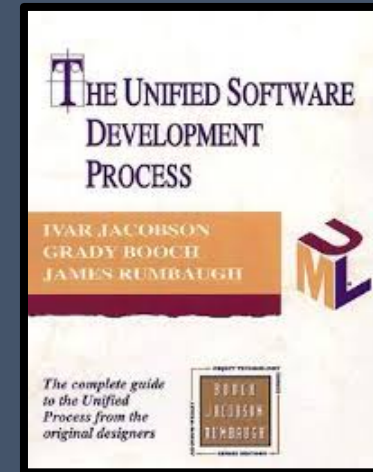
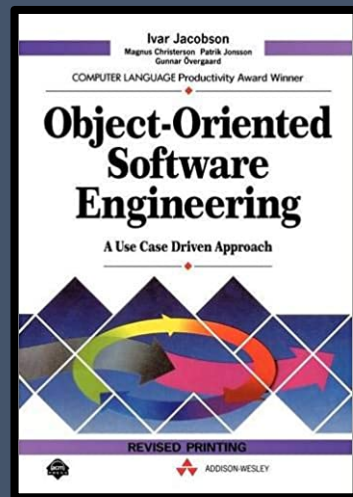
From Requirements model to Design model

- There are some approaches to ease bridging the gap between the Requirements model and a Design model.
 - ✓ Larman's System Sequence Diagram and GRASP principles.
 - ✓ Ivar Jacobson's Robustness analysis.
- We use Robustness analysis in this course.



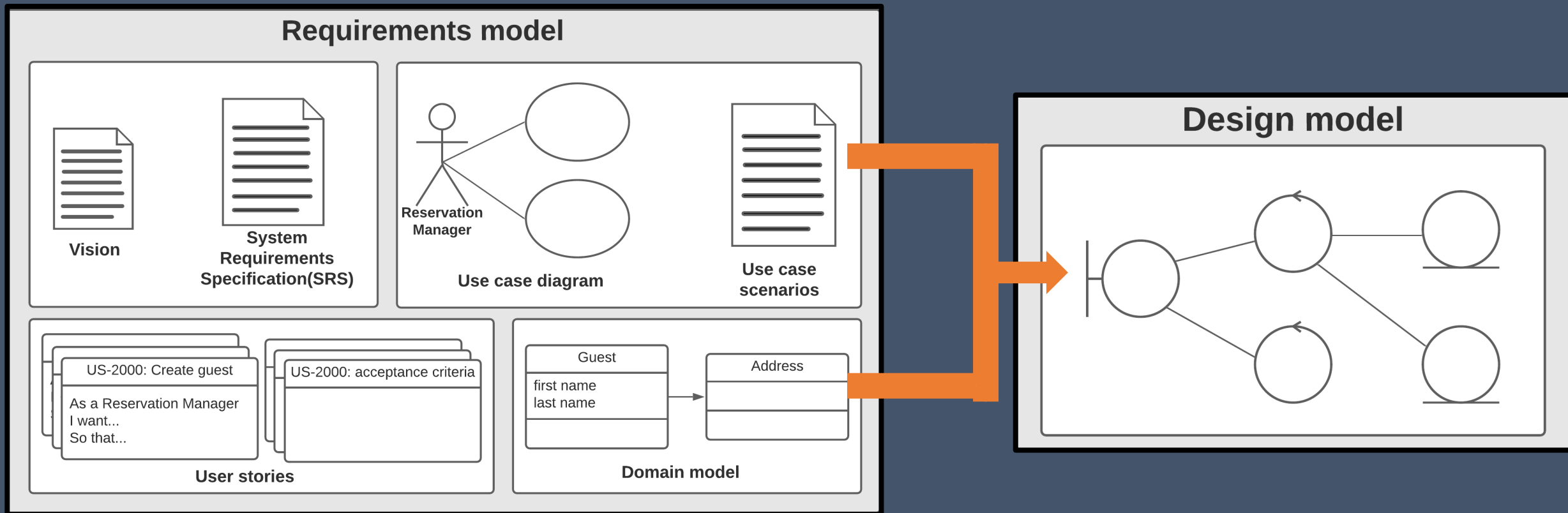
Where does robustness analysis come from?

- **Robustness analysis is not part of UML.**
 - ✓ The practice originated with Ivar Jacobson's Objectory method.
- It was a famous design technique described at some books at 90's.



Robustness analysis

- Process that leads from a **Requirements model** to a **Design model**.



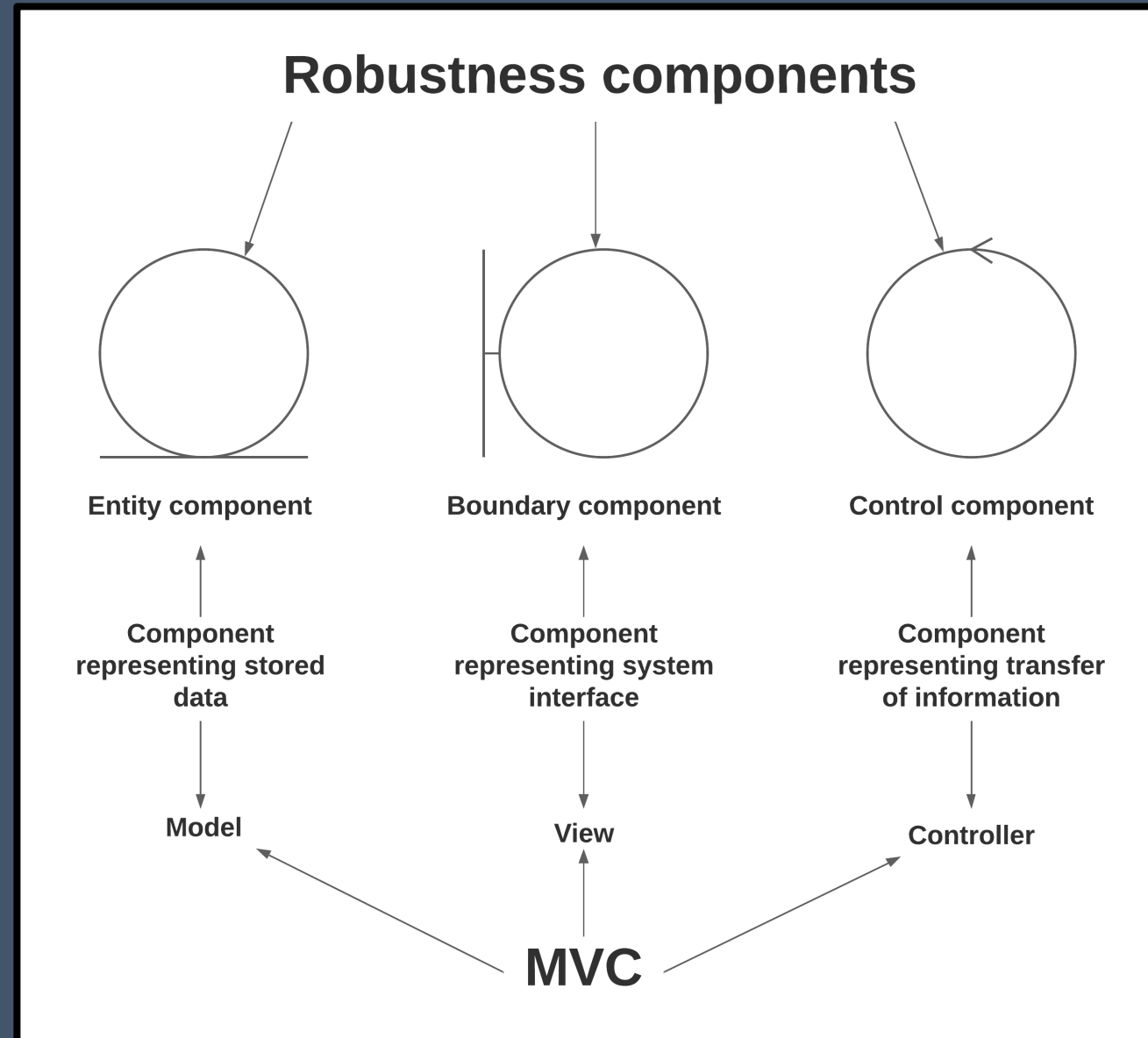
Robustness analysis

- Involves **analyzing the narrative text** of use cases in order to:
 - ✓ **Identify the set of objects** that will **participate** in the **use cases**.
 - ✓ **Classifying the objects** based on the **roles they play**.

Use case scenario: Create a Guest
Pre-condition(s): - A screen with an option "<i>Create new Guest</i>" is visible. - The Reservation Manager is authenticated. Main flow: 1.The Reservation Manager selects the option "<i>Create new Guest</i>". 2.The system show a form requesting Guest data. 3.The Reservation Manager enters the form data. 4.The systems shows a message "<i>Guest created sucessfully</i>". Alternate flow(s):

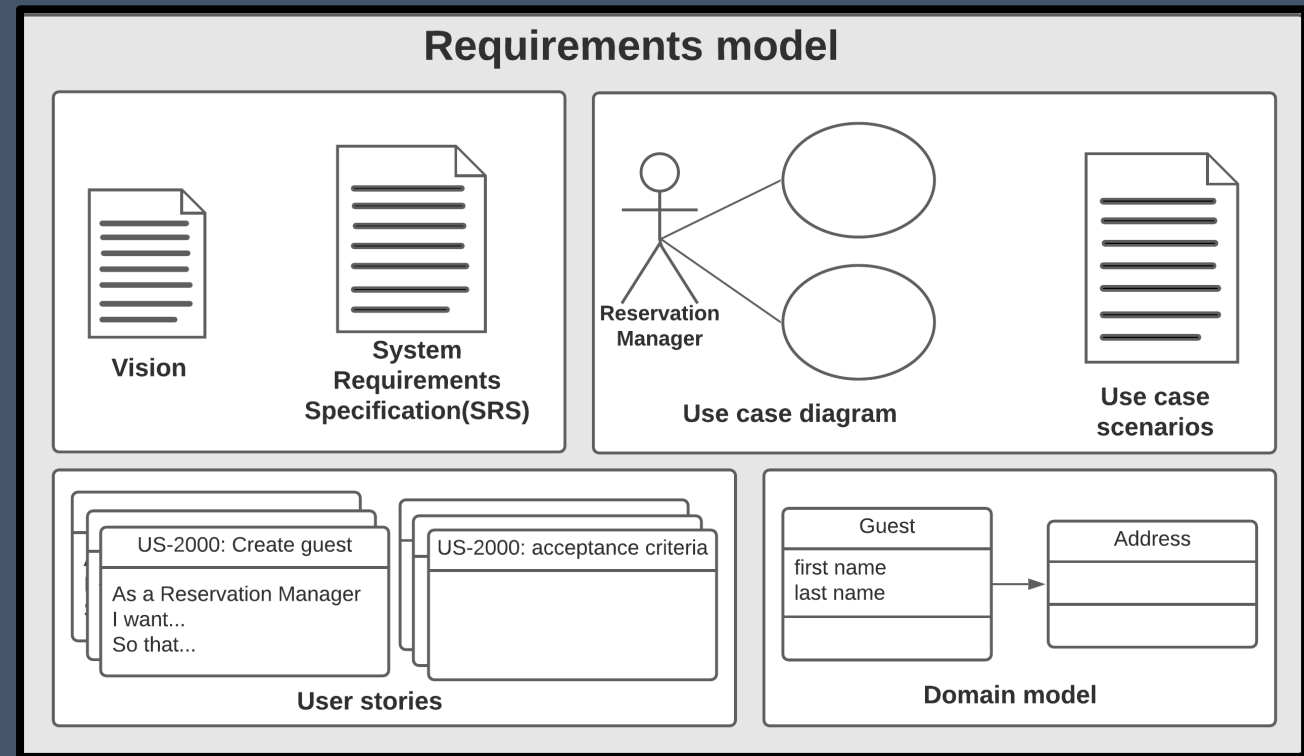
Robustness analysis

- Helps you to **bridge** the gap from **Use Cases** and **Domain Classes**, and the **Model-View-Control (MVC)** architecture pattern.
 - ✓ We are going to discuss **MVC** in a later lecture.



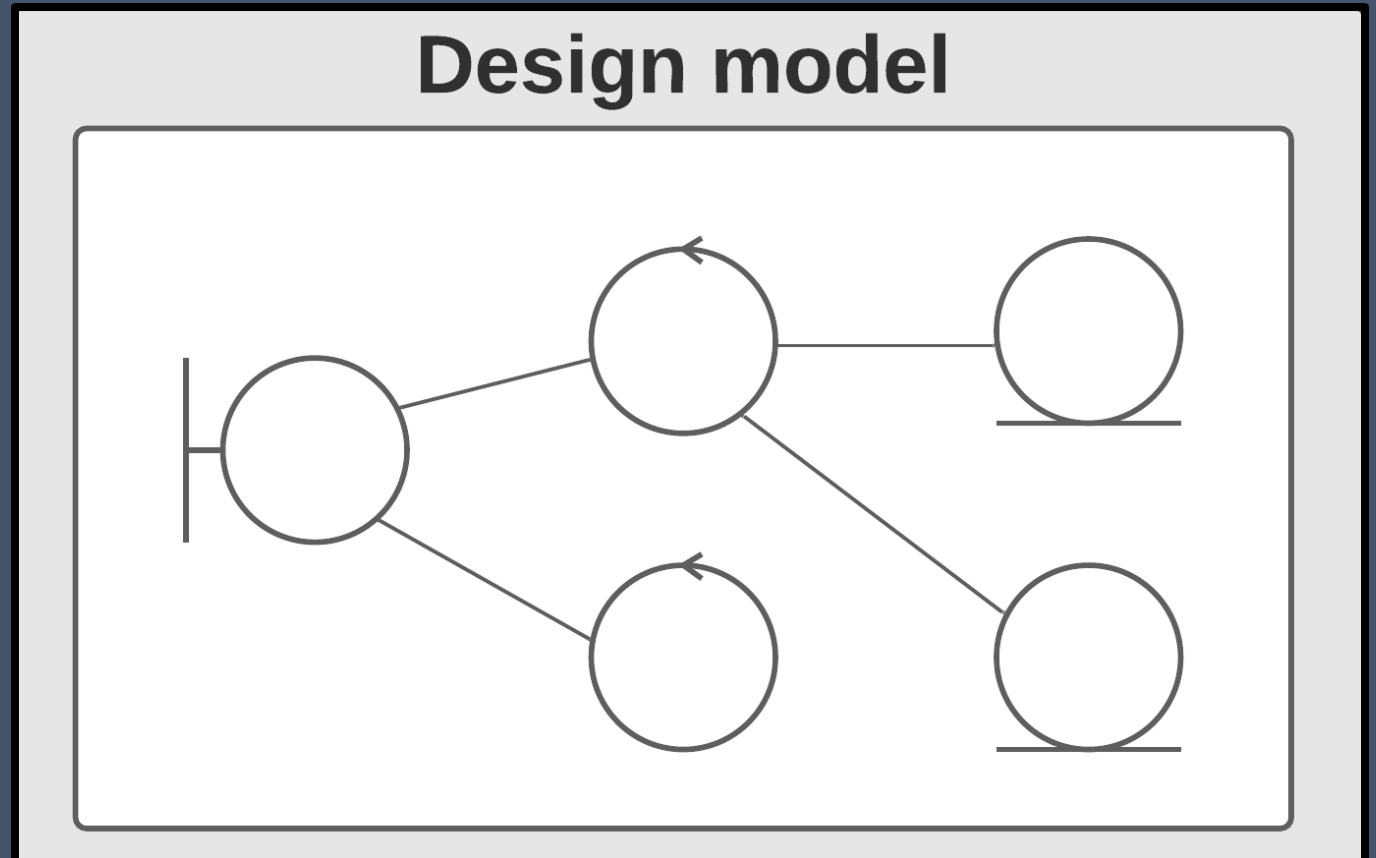
Robustness analysis: inputs

- A use case and its scenarios.
 - ✓ Main scenario
 - ✓ Alternative scenarios
- A use case Activity diagram for that use case.
 - ✓ If available.
- The Domain model.
- A user story (optional)



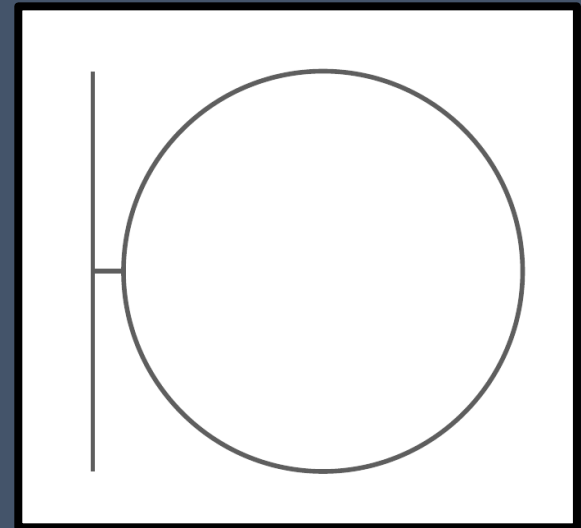
Robustness analysis: output

- The output of a robustness analysis is a Design model.



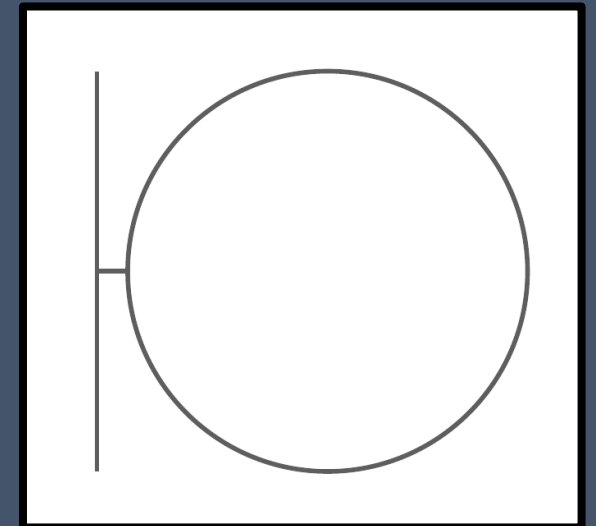
Boundary components

*“A **boundary class (component)** is used to model interaction between the **system** and its **actors** (that is, **users** and **external systems**).”*



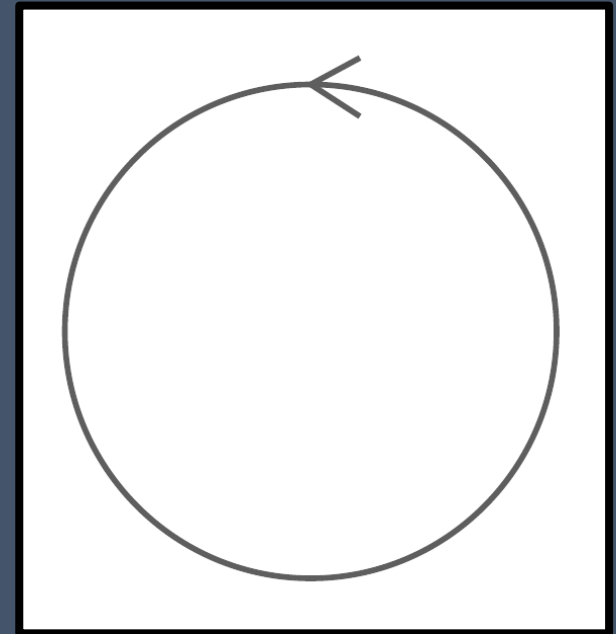
Boundary components

- **Abstractions of UI screens, sensors, communication interfaces, and so on.**
- **High-level UI components.**
- **Every boundary component must be associated with at least one actor.**



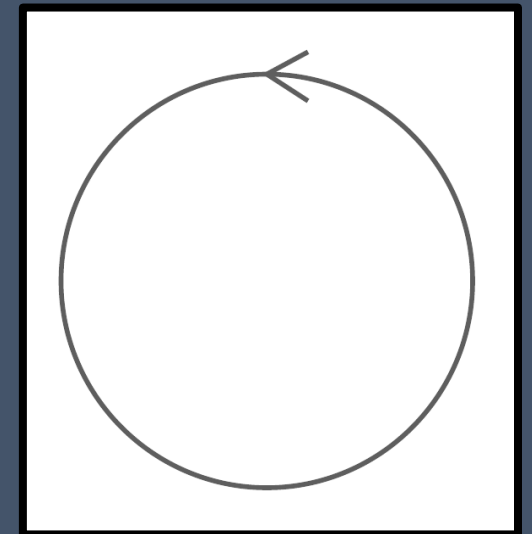
Control (service) components

“Control (service) classes (components) represent coordination, sequencing, transactions, and control of other objects and are often used to encapsulate control related to a specific use case.”



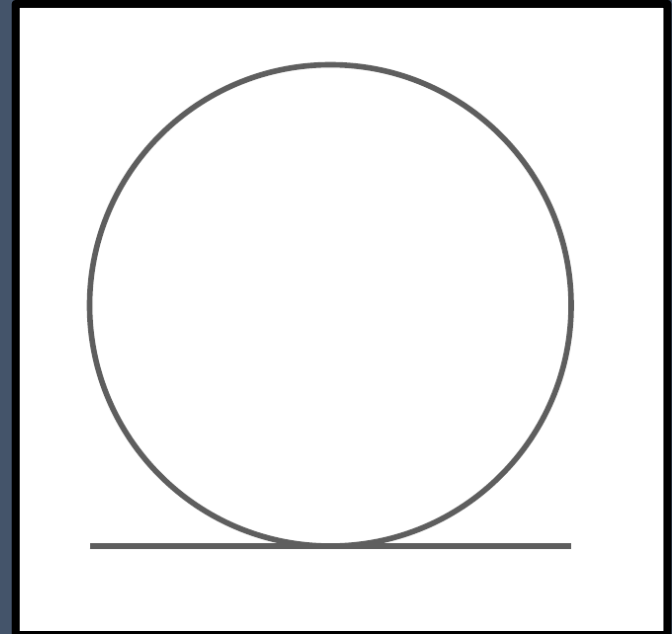
Control (service) components

- Known as controllers in MVC.
- Coordinate control flow.
- Isolate any changes in workflow from the boundary and entity components.



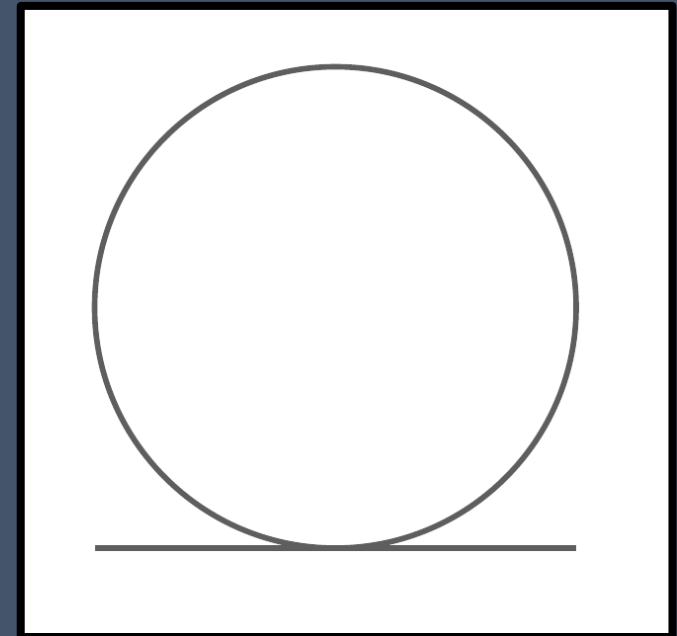
Entity components

*“An entity class (component) is used to model information that is long-lived and often **persistent**.”*



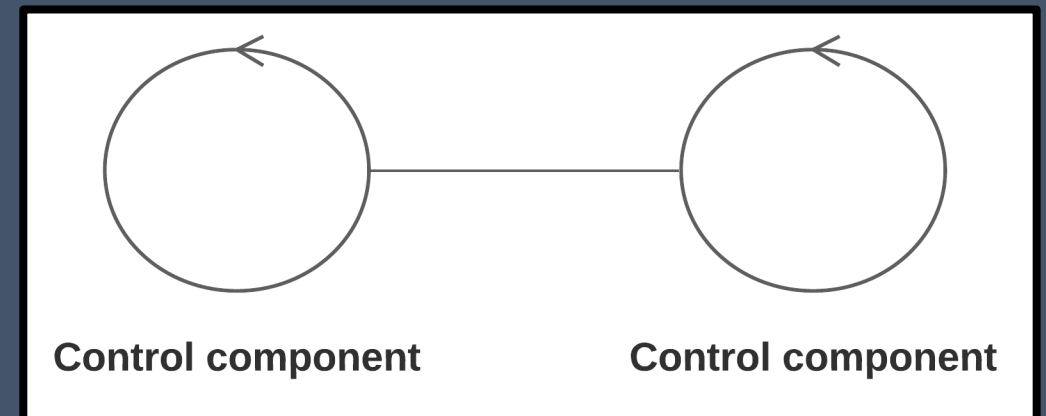
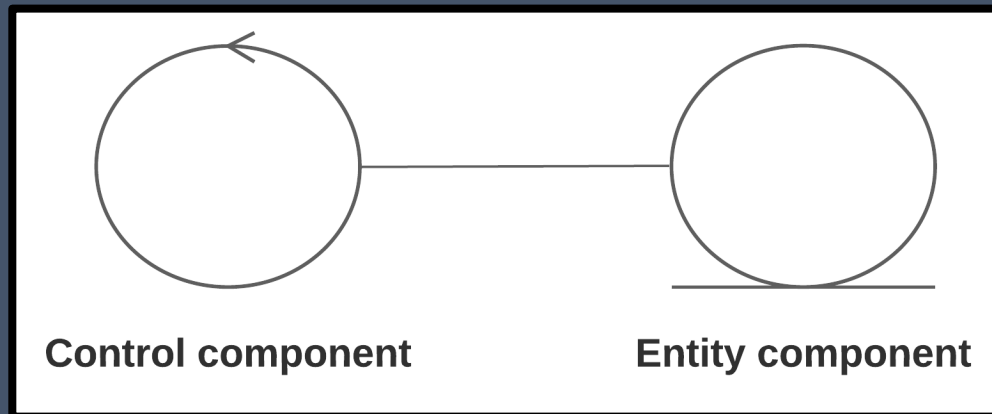
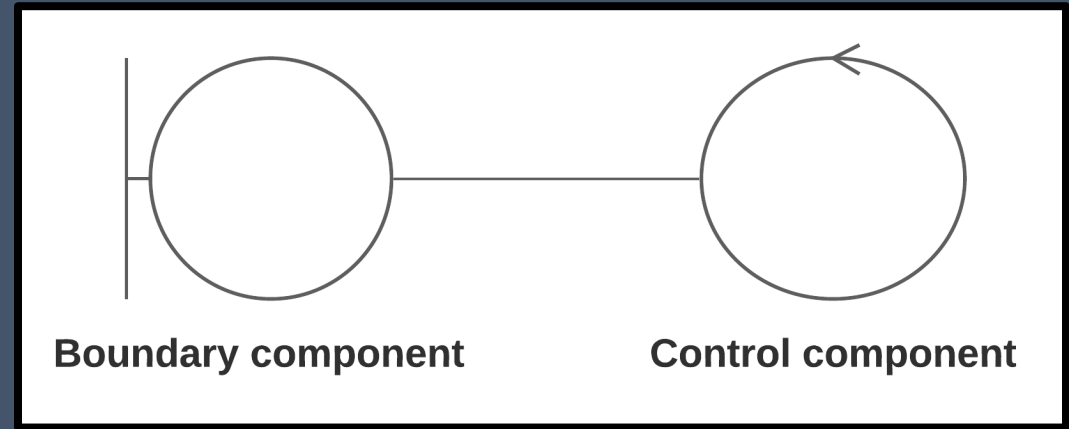
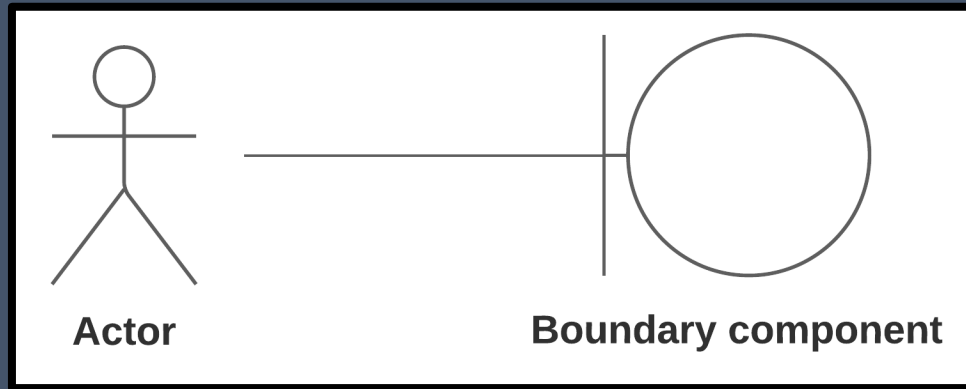
Entity components

- **Entities** usually **correspond** to **domain objects**.
- **Most entities** are **persistent**.
- **Entities** can have **complex behavior**.

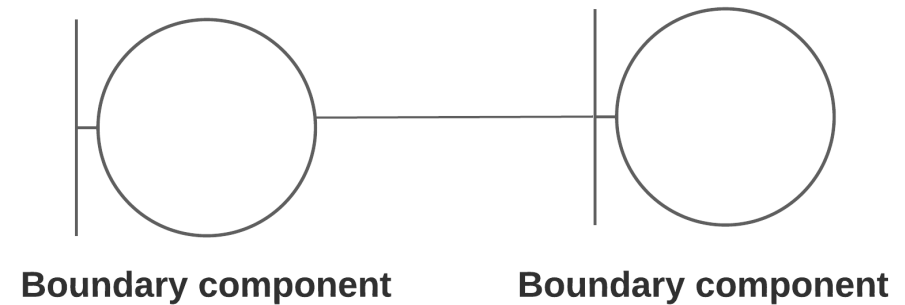
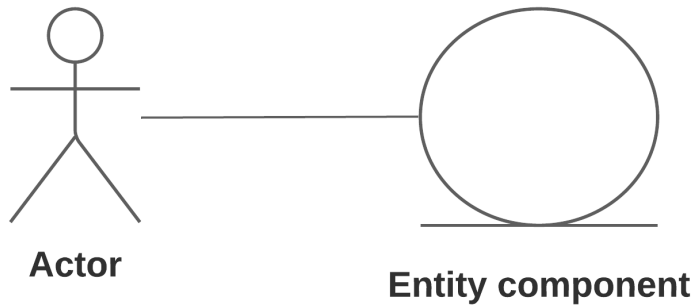
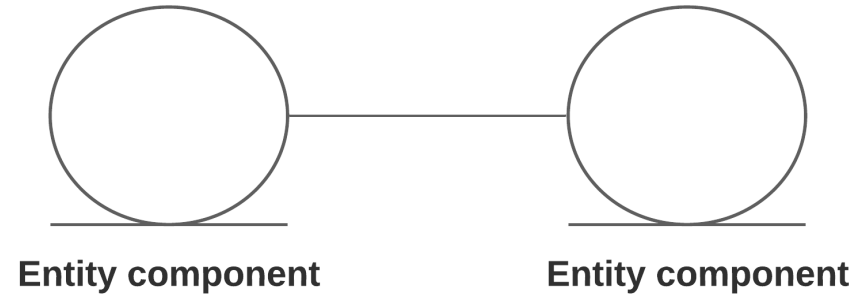
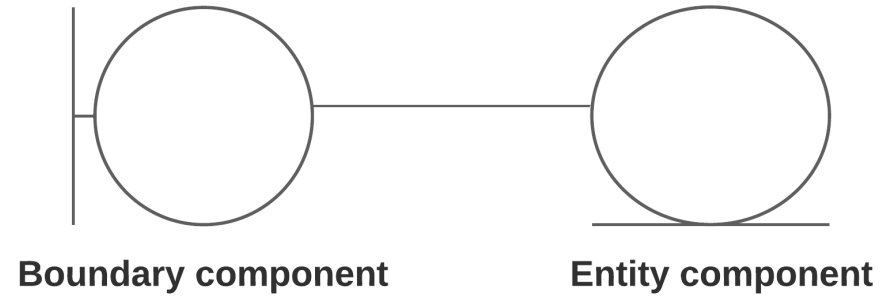
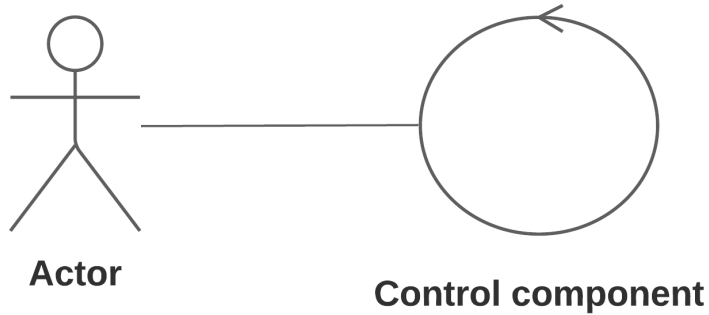


Connection rules during robustness analysis

Valid connection rules



Invalid connection rules



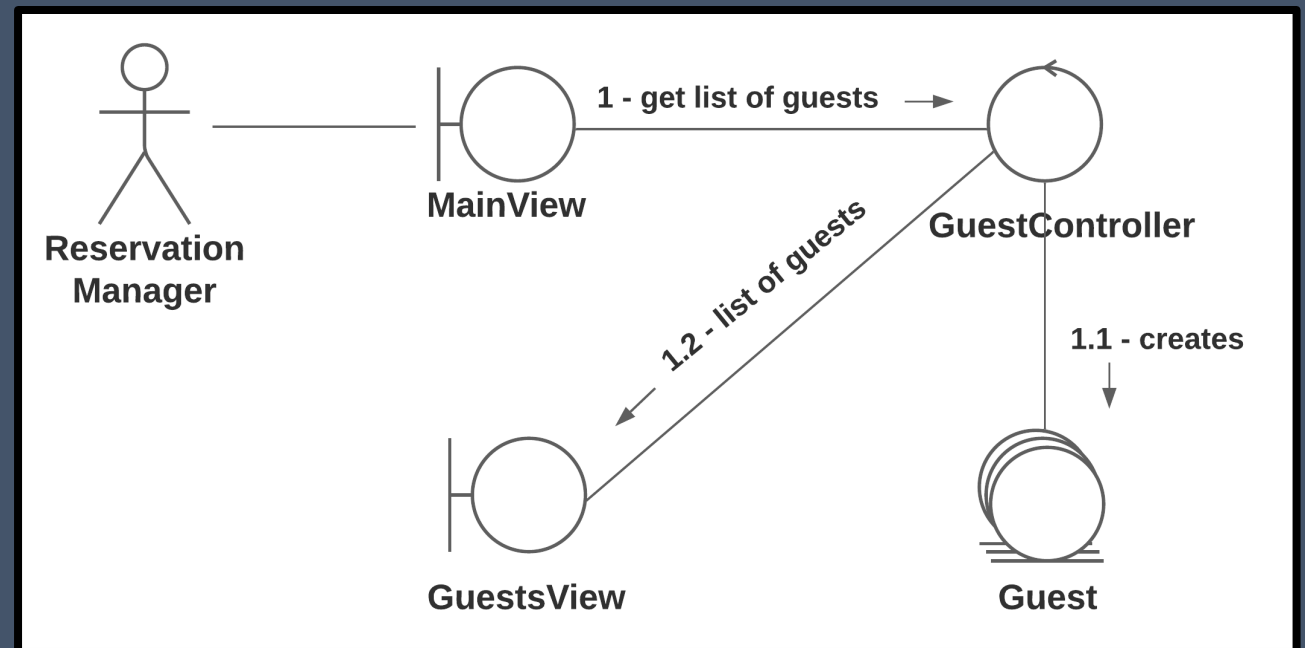
Design model & the UML Collaboration diagram

Lecture: Design model & the UML Collaboration diagram

Module: Software development as model transformations

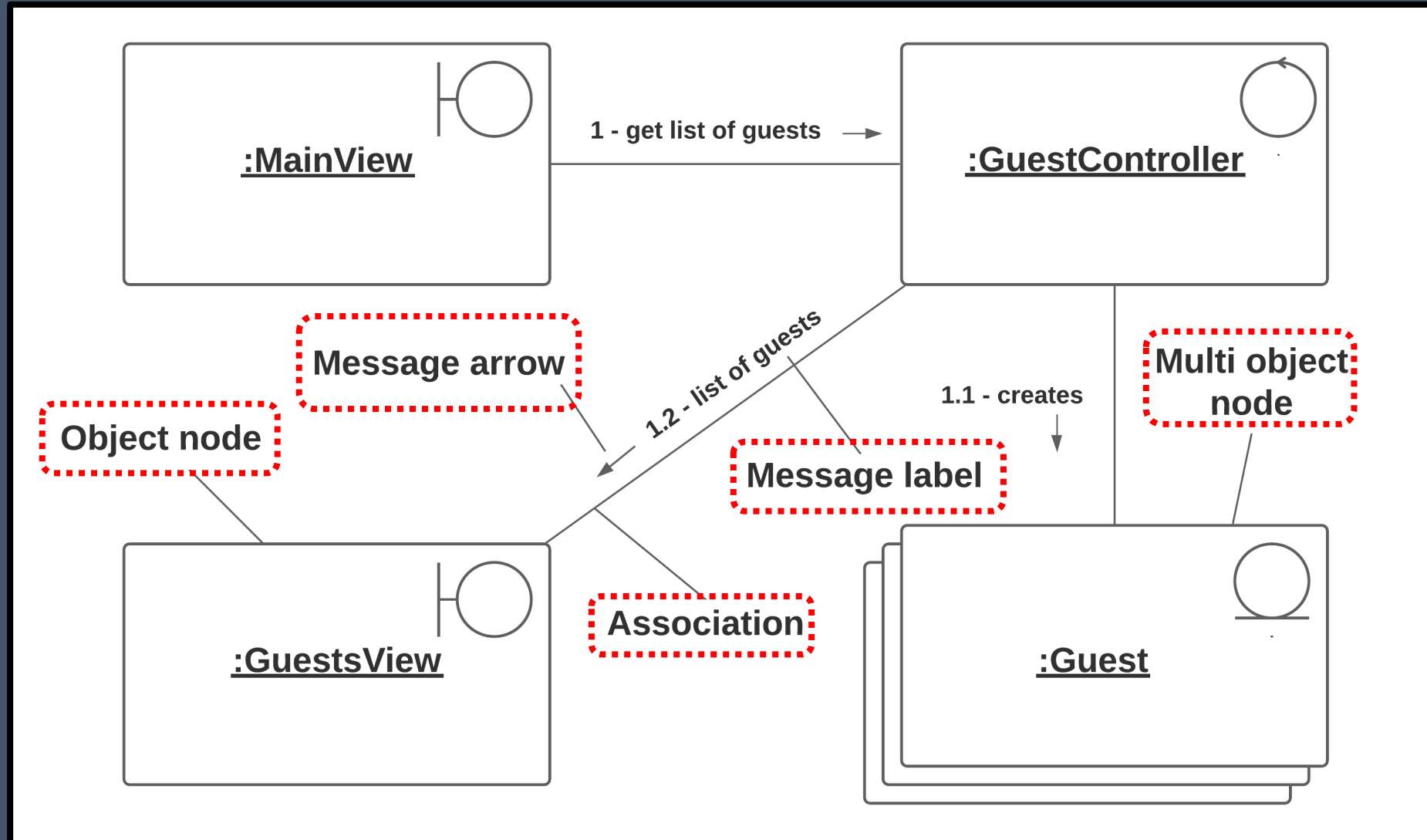
Design model & UML Collaboration diagram

- The **Design model** is usually captured as a **UML Collaboration diagram** with design components:
 - ✓ **Boundary**
 - ✓ **Service (control)**
 - ✓ **Entity**

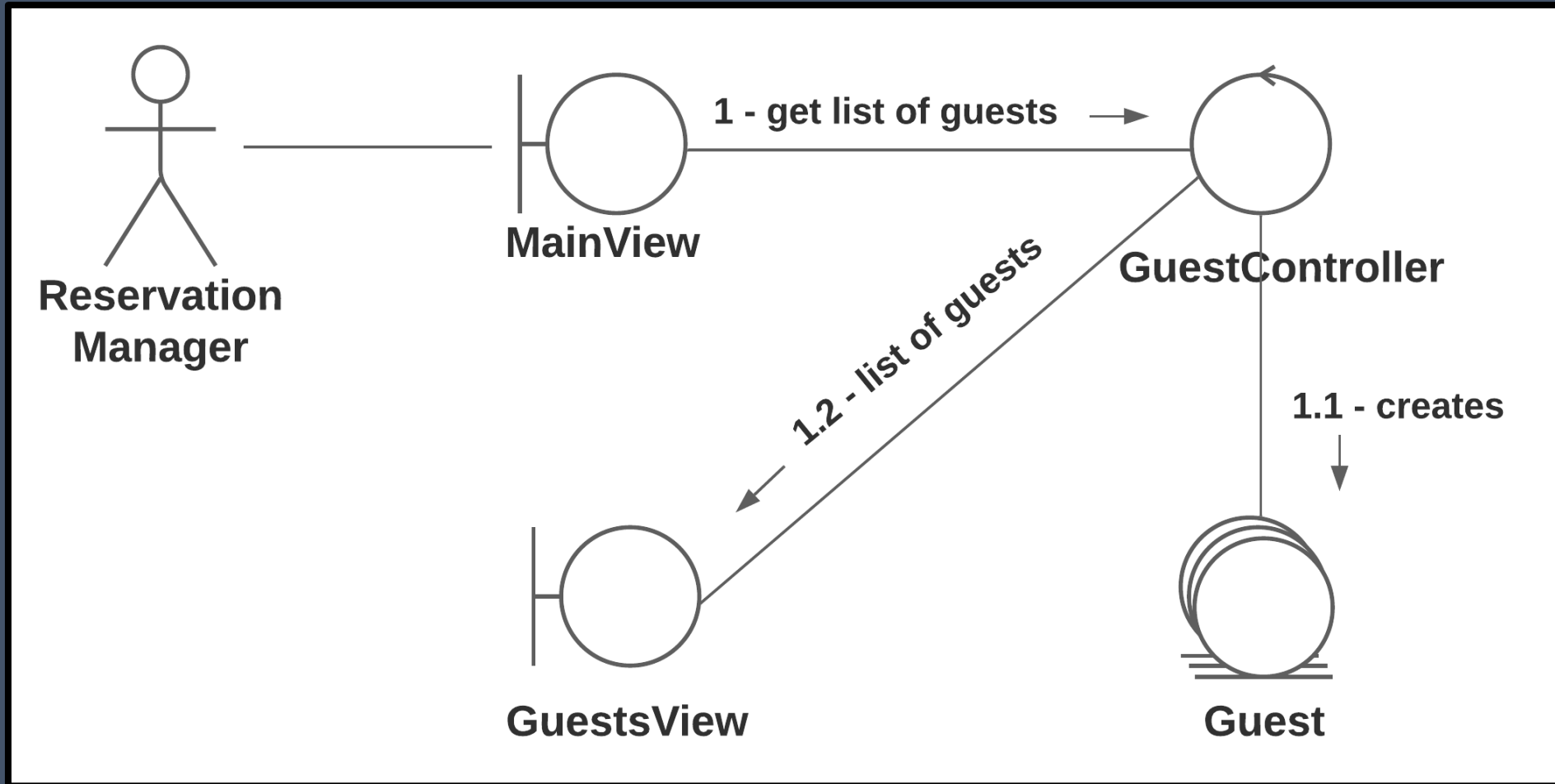


Elements of a UML Collaboration diagram

Elements of a UML Collaboration diagram



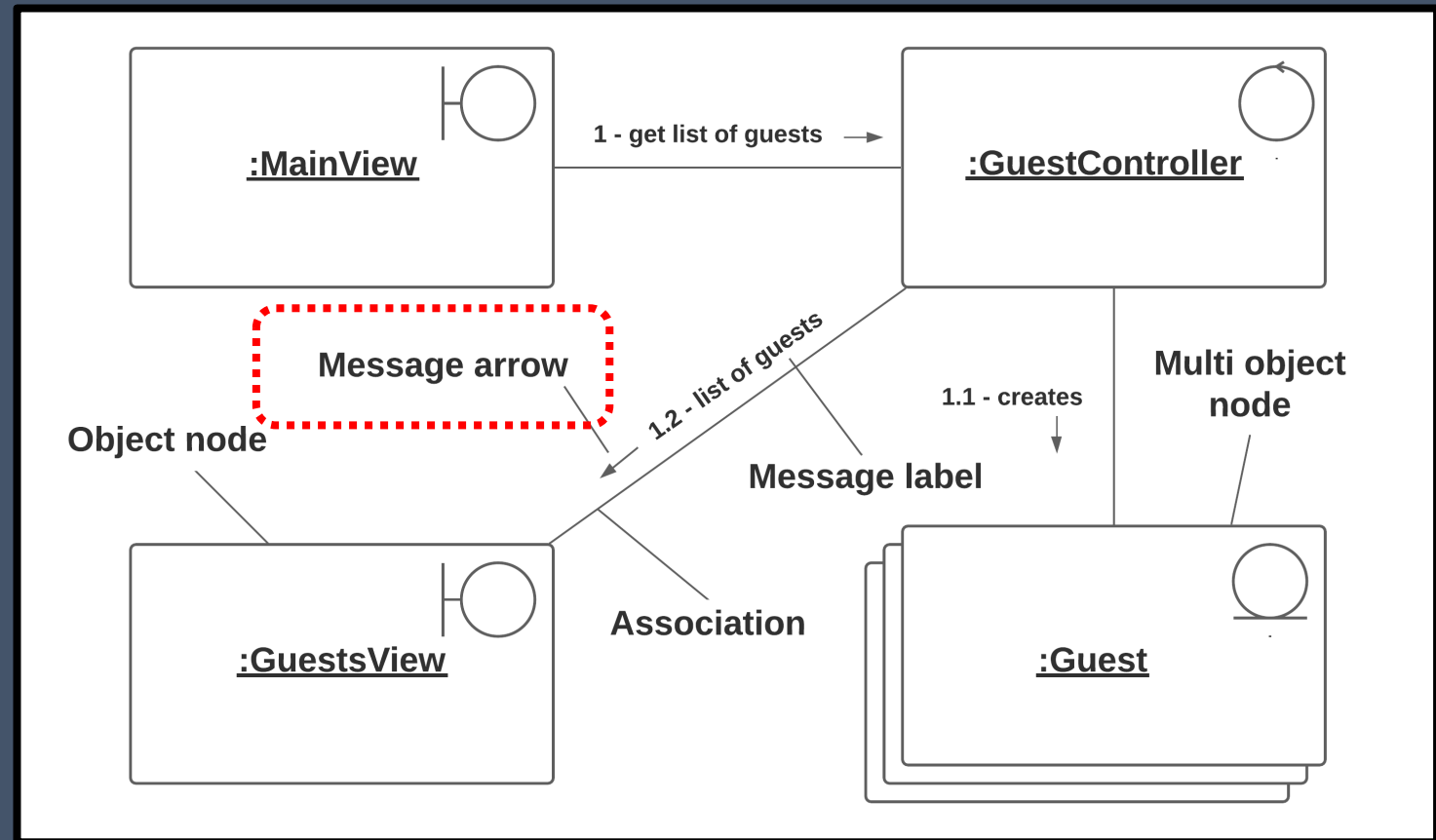
A variation of the UML collaboration diagram



Elements of a UML Collaboration diagram

Message arrows can indicate:

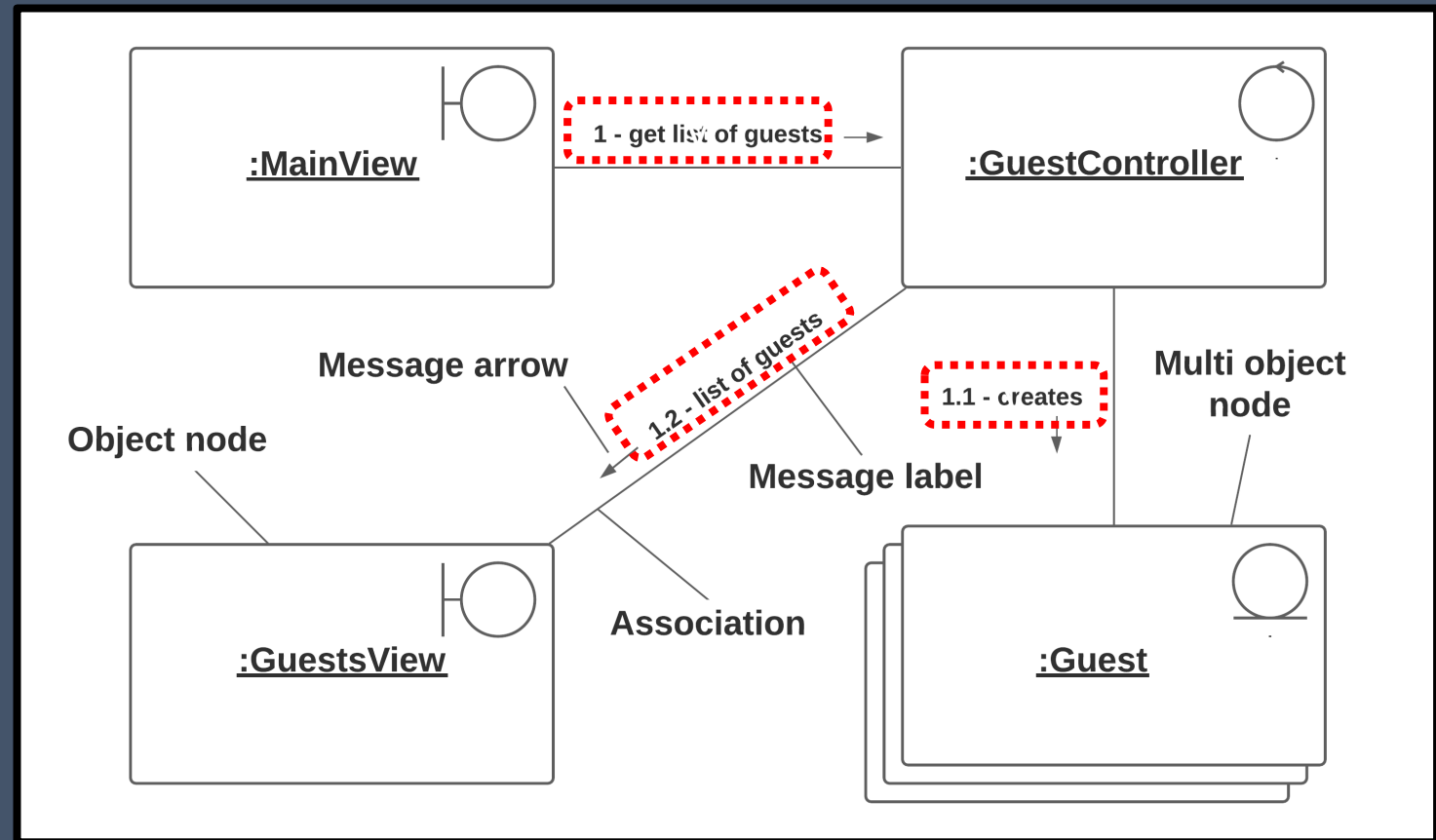
- ✓ A method call
- ✓ Remote method invocation
- ✓ An asynchronous message



Elements of a UML Collaboration diagram

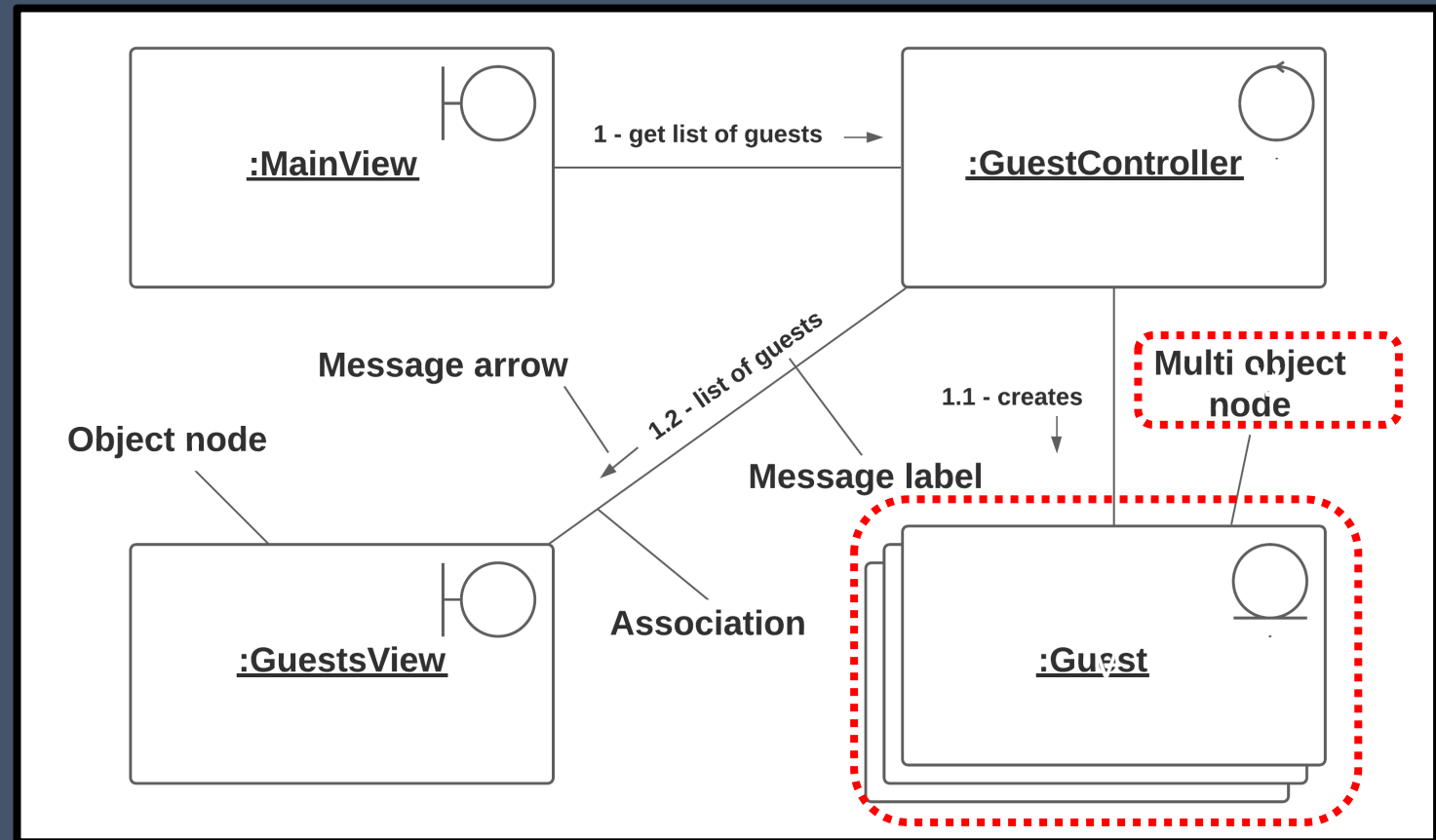
Message (sequence) labels indicate:

- ✓ The order of the message.
- ✓ The activity the message will invoke.



Elements of a UML Collaboration diagram

Multi-objects represent a collection of related objects.



Questions?